

Codefest 5: Solutions

Hardware for Artificial Intelligence and Machine Learning (HW4AI)

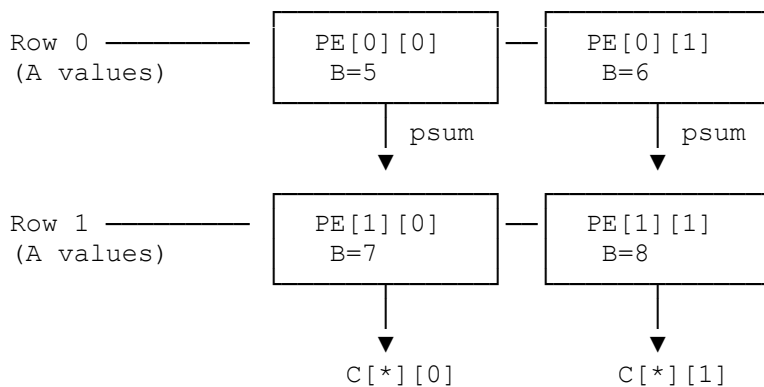
ECE 410/510

Spring 2026

CMAN solutions — Systolic array trace (weight-stationary)

(a) PE diagram

Below is the 2×2 array. Each PE holds one element of B (stationary). A values stream in from the left; partial sums propagate downward and emerge at the bottom edge as the elements of C.



(b) Cycle-by-cycle trace

Input schedule. Row 0 of the array receives column 0 of A (i.e., $A[0][0]=1$ then $A[1][0]=3$) over cycles 1–2. Row 1 receives column 1 of A ($A[0][1]=2$ then $A[1][1]=4$) over cycles 2–3, skewed by one cycle to align with the diagonal propagation of partial sums.

Notation. PE columns show the multiply-accumulate result that PE produces in that cycle (the value sent down to the row below). Dashes indicate idle PEs. The Output column shows which element of C emerges from a column at the bottom edge that cycle.

Cycle	In row 0	In row 1	PE[0][0]	PE[0][1]	PE[1][0]	PE[1][1]	Output
1	$A[0][0]=1$	—	5	—	—	—	—
2	$A[1][0]=3$	$A[0][1]=2$	15	6	$5 + 2 \cdot 7 = 19$	—	$C[0][0] = 19$
3	—	$A[1][1]=4$	—	18	$15 + 4 \cdot 7 = 43$	$6 + 2 \cdot 8 = 22$	$C[1][0] = 43, C[0][1] = 22$
4	—	—	—	—	—	$18 + 4 \cdot 8 = 50$	$C[1][1] = 50$

Verification: $C[0][0] = 1 \cdot 5 + 2 \cdot 7 = 5 + 14 = 19$. $C[0][1] = 1 \cdot 6 + 2 \cdot 8 = 6 + 16 = 22$. $C[1][0] = 3 \cdot 5 + 4 \cdot 7 = 15 + 28 = 43$. $C[1][1] = 3 \cdot 6 + 4 \cdot 8 = 18 + 32 = 50$. All four match the expected $C = \begin{bmatrix} 19, 22 \\ 43, 50 \end{bmatrix}$.

(c) Operation and memory-access counts

(c.1) Total MAC operations. Each output element $C[i][j]$ is a dot product of length $K=2$, so it requires 2 MACs. With 4 outputs, total MACs = $4 \times 2 = 8$.

(c.2) Input reuse counts.

- B values (held stationary in PEs): each $B[k][j]$ participates in producing one column of C. With $M=2$ rows of A streaming through, each B value is reused 2 times. Each of the 4 B values: reuse count = 2.
- A values (streamed): each $A[i][k]$ enters one row of the array (row k in this layout) and propagates rightward across $N=2$ PEs in that row, multiplying with $B[k][0]$ then $B[k][1]$. Each A value is reused 2 times.

(c.3) Off-chip memory accesses.

- Reads of A: 4 distinct values ($M \cdot K = 2 \cdot 2$), each fetched once from off-chip and streamed into the array. = 4 reads.
- Reads of B: 4 distinct values ($K \cdot N = 2 \cdot 2$), loaded once into the PEs at startup and held. = 4 reads.
- Writes of C: 4 distinct values ($M \cdot N = 2 \cdot 2$), each emitted once at the bottom of a column. = 4 writes.
- Total off-chip accesses = $4 + 4 + 4 = 12$.

Compare to a naive scalar implementation: each MAC would touch operands from off-chip, giving $2 \cdot 8 = 16$ reads for the operands plus 4 writes = 20 accesses. The systolic array achieves the same result with 12 accesses by holding B stationary and reusing each operand twice on-chip.

(d) Output-stationary variant

If this were output-stationary, each PE would hold a fixed accumulator for one element of C, so $PE[i][j]$ would keep $C[i][j]$ resident and accumulate into it as the matching A and B values stream past.

Common errors and grading notes

Acceptable variations: students may swap which axis is the input row versus the partial-sum direction, may load A (transposed) instead of B, or may use a different cycle-skew convention. Any trace that produces the correct $C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$ with consistent MAC arithmetic at each PE earns full credit on parts (a)–(c). Award credit on (b) for any trace of at least 4 cycles that shows partial-sum propagation between PEs and arrives at the correct outputs.

- Common mistake 1: ignoring the cycle skew between rows, which causes partial sums to accumulate the wrong A·B pairings (e.g., producing $1 \cdot 5 + 2 \cdot 8 = 21$ instead of 19 for $C[0][0]$).
- Common mistake 2: counting MAC operations as $M \cdot N \cdot K = 8$ but listing reuse as 1 (treating the systolic array as if it streamed every operand from memory). The whole point of the stationary dataflow is the reuse count of K (or M, depending on which operand is stationary).
- Common mistake 3: counting the partial-sum register transfers between PEs as off-chip memory accesses. On-chip register-to-register transfers do not count as off-chip accesses.
- Common mistake 4 (output-stationary part): answering “the inputs A and B” instead of “the partial sums / output elements $C[i][j]$.” The defining feature of output-stationary is that the output accumulator is what stays put.